UNITY GAME DESIGN DOCUMENT

2D 타워 디펜스

2D Defence Game

"길은 코드가 아니라 그림이 정한다.
타일을 어디에 칠하느냐가 그대로 레벨 디자인이 된다."

장르	2D 타워 디펜스 · 싱글플레이
엔진 / 버전	Unity 6.3 LTS (6000.3.10f1) · Universal 2D
원 교재	한빛아카데미『유니티 입문』14장「실전 2D 게임 프로젝트」
문서 작성	원광대학교 게임콘텐츠학과
작성 근거	출판사 제공 PPT 65쪽 전수 분석 + C# 스크립트 9개 직접 검토
버전 / 작성일	v1.0 / 2026. 10. 13.

목차

0. 문서 정보 4
1. 개요 (Overview) 4
    1-1. 이 게임이 가르치는 것 5
    1-2. 교재가 제시한 6요소와 실제 구현 범위 5
2. 코어 게임플레이 7
    2-1. 플레이어가 내리는 결정 7
    2-2. 승리와 패배 8
3. 레벨과 맵 9
    3-1. 타일맵 세 장의 역할 9
    3-2. 맵 설계도 9
    3-3. 좌표계와 단위 11
4. 길 찾기 시스템 12
    4-1. 베이크 순서 12
    4-2. 경로가 결정되는 방식 13
5. 적과 웨이브 14
    5-1. 적의 생애 14
    5-2. 도착 판정 14
    5-3. 웨이브 데이터 구조 14
    5-4. 웨이브 전환 규칙 15
6. 타워와 발사체 16
    6-1. 타워 건설 16
    6-2. 타워의 공격 16
    6-3. 발사체 17
7. 경제와 생존 19
    7-1. 설계상의 비대칭 19
8. 데이터 명세와 밸런싱 20
    8-1. 전체 수치표 20
    8-2. 밸런싱 검토 — 지금 수치로 계산해 보면 20
    8-3. 조정안 21
9. 씬 구성과 오브젝트 명세 23
    9-1. 하이어라키 23
    9-2. 프리팹 3종 23
    9-3. 인스펙터 연결 23
10. 스크립트 아키텍처 25
    10-1. 파일 목록 25
    10-2. 실행 순서 26
    10-3. 구조의 특징과 한계 26
11. 개발 단계와 체크포인트 27
12. 알려진 제약과 미구현 28

12-1. 미구현 기능 28
12-2. 교재 원본 코드의 결함 28
12-3. 기술적 제약 29
13. 확장 과제 30
13-1. 추천 — 먼저 할 것 세 개 30
14. 참고 자료 32

0. 문서 정보

이 문서는 한빛아카데미 『유니티 입문』 14장의 2D 타워 디펜스 프로젝트를 게임 디자인 문서(GDD) 형식으로 정리한 것입니다. 교재는 "따라 만드는 순서"로 서술되어 있어 게임이 어떤 규칙으로 돌아가는지가 지면 곳곳에 흩어져 있습니다. 이 문서는 그 규칙을 한 자리에 모으고, 수치를 표로 묶고, 스크립트 사이의 호출 관계를 그림으로 드러내는 것을 목적으로 합니다.

근거 자료	내용	반영 절
출판사 제공 PPT	65쪽 전체. 오브젝트 생성 수치, 컴포넌트 설정, 인스펙터 연결, 코드 14-1 ~ 14-8	3 ~ 9
C# 스크립트	Enemy · EnemyMove · EnemyMgr · Tower · Projectile · TowerMgr · GameMgr · ForceNavMove · SpawningPrefab — 9개 파일 직접 검토	5 ~ 10
실제 제작 과정	타일맵 작성, NavMesh 베이크, 컴파일 오류 해결 과정에서 확인된 동작	4, 11, 12

이 문서가 수치를 적는 원칙

· 모든 수치는 스크립트의 초기화 값이나 PPT에 명시된 값에서 직접 가져왔다. 어느 쪽에도 없는 값은 "교재에 없음"이라고 표시하고, 이 문서가 제안하는 값은 "제안"이라고 따로 밝혔다.

· 교재 코드에는 빠진 부분과 오류가 상당수 있다. 이 문서는 그것을 보완한 상태를 기준 사양으로 삼되, 12절에 원본의 결함을 따로 정리했다.

· 8절의 밸런싱 검토는 실제 수치를 넣어 계산한 결과다. 현재 값으로는 패배가 성립하지 않는다는 점이 거기서 드러난다.

1. 개요 (Overview)

제목	2D 타워 디펜스 (2D Defence Game)
한 문장 요약 (High Concept)	적이 정해진 길을 따라 목표 지점으로 끊임없이 걸어온다. 플레이어는 길 옆 빈 터를 클릭해 타워를 세우고, 모은 골드로 방어선을 넓혀 가며 마지막 웨이브까지 버텨 낸다.
핵심 동사	본다 → 고른다 → 세운다 → 버틴다

장르	2D 탑다운 타워 디펜스 (고정 경로형 · Fixed Path Tower Defense)
플레이 인원 / 시점	1인 / 2D 탑다운 고정 카메라
조작	마우스 좌클릭 하나 — 타일을 클릭하면 타워가 선다 (New Input System)
한 판 길이	웨이브 3개 기준 약 2 ~ 3분
대상 플랫폼	PC (WebGL 빌드로 브라우저 배포 가능)
엔진 / 패키지	Unity 6.3 LTS · NavMeshPlus(h8man) · AI Navigation · Input System
아트 에셋	Pixel Art Top Down – Basic (TX Tileset Grass)
대상 학습자	유니티 에디터 조작과 C# 기초 문법을 익힌 2학년 수준. 타일맵 · 내비메시 · 프리팹 · 매니저 패턴을 한 프로젝트에서 엮어 보는 단계

1-1. 이 게임이 가르치는 것

이 프로젝트는 "타워 디펜스를 만드는 법"보다 "유니티의 세 가지 시스템을 한 게임 안에서 맞물리게 하는 법"을 가르칩니다. 세 가지는 각각 다음과 같습니다.

- **타일맵 — 그림이 곧 규칙이다:** 경로 탐색 알고리즘을 한 줄도 짜지 않는다. 돌길 타일을 어디에 칠하느냐가 그대로 적의 이동 경로가 되고, 초록 타일을 어디에 칠하느냐가 그대로 타워 건설 가능 구역이 된다. 레벨 디자인이 코딩이 아니라 그리기가 되는 경험이다.
- **2D NavMesh — 3D 기술을 2D 로 눕히기:** 유니티의 내비메시는 본래 3D 의 XZ 평면용이다. NavMeshPlus 가 평면을 −90 도 눕혀 XY 에 맞추고, 타일맵에서 모양을 읽어 베이크 재료로 넘긴다. 엔진 기능의 전제를 이해하고 우회하는 연습이 된다.
- **매니저 패턴 — 누가 무엇을 책임지는가:** 적을 만드는 일, 타워를 세우는 일, 골드와 HP 를 세는 일이 각각 다른 클래스에 있다. 적을 찾는 일은 EnemyMgr 한 곳에만 있어서 타워가 몇 개로 늘어도 탐색 코드는 하나뿐이다. 책임을 나누는 설계의 첫 경험이다.

1-2. 교재가 제시한 6 요소와 실제 구현 범위

교재 01절은 타워 디펜스의 구성요소로 여섯 가지를 듭니다. 이 중 실제로 구현되는 것은 넷입니다.

구성요소	구현 여부	담당
맵 시스템	구현	Tilemap 3종 (BgTilemap · ObjTilemap · Building)
길 찾기	구현	NavMeshPlus + NavMeshAgent

구성요소	구현 여부	담당
웨이브 로직	구현	EnemyMgr.waves[]
공격 메커니즘	구현	Tower + Projectile
업그레이드 시스템	미구현 — 13절 확장 과제	없음
스테이지 클리어 로직	부분 — 승패 판정만	GameMgr / EnemyMgr.CompleteGame()

2. 코어 게임플레이

이 게임에는 두 개의 루프가 동시에 돕니다. 하나는 시스템이 자동으로 돌리는 적의 루프이고, 다른 하나는 사람이 손으로 돌리는 플레이어의 루프입니다. 두 루프는 골드와 HP라는 자원 두 개로 맞물립니다.

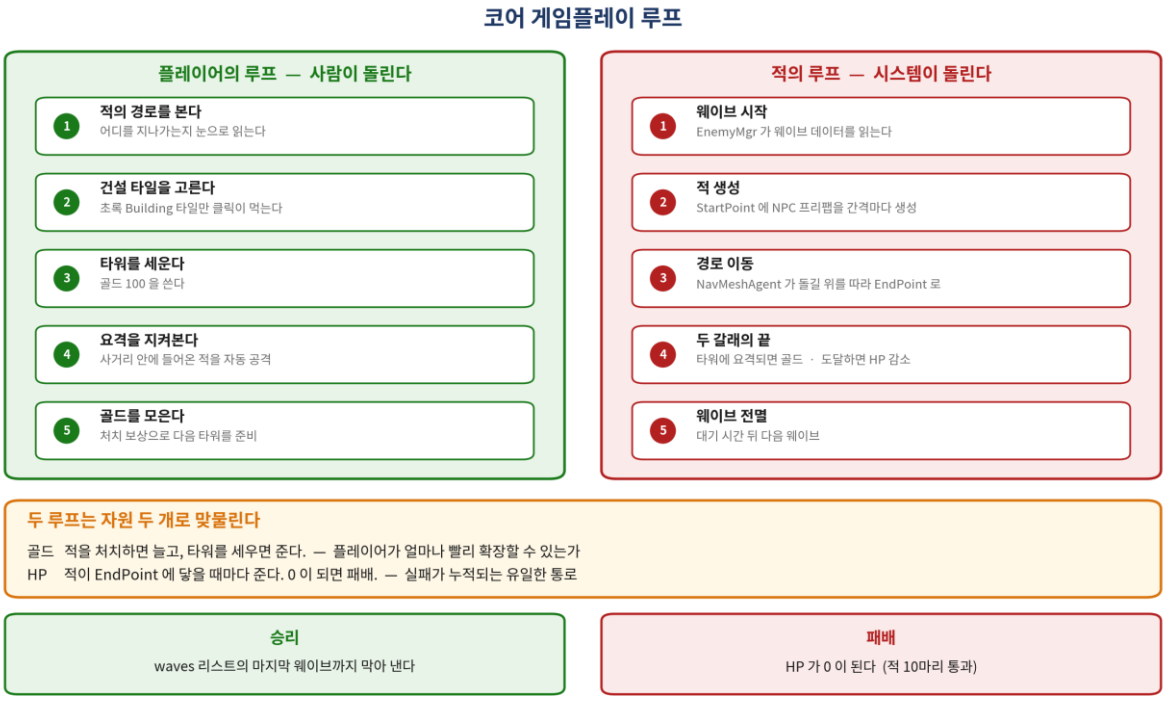


그림 2-1 코어 게임플레이 루프

2-1. 플레이어가 내리는 결정

조작은 마우스 클릭 하나뿐입니다. 그래서 이 게임의 재미는 "어떻게 조작하는가"가 아니라 "어디에 세우는가"에서 나옵니다. 플레이어가 매 순간 저울질하는 것은 다음 셋입니다.

결정	저울질하는 것	잘못 고르면
어디에 세울 것인가	사거리 3 안에 길이 얼마나 길게 들어오는가. 길이 꺾이는 안쪽 모서리가 가장 길게 걸린다	사거리 밖이라 한 발도 못 쏜다
지금 세울 것인가 모을 것인가	타워 1개 = 골드 100. 지금 세우면 당장 막지만, 아끼면 더 좋은 자리를 선점할 수 있다	웨이브가 몰려올 때 골드가 비어 있다
어디를 포기할 것인가	HP 100 은 적 10마리를 흘려보낼 여유다. 전부 막지 않아도 된다	모든 적을 막으려다 자원이 분산된다

2-2. 승리와 패배

조건	판정 위치	처리
승리	EnemyMgr — curWaveIdx 가 waves.Count 에 도달	CompleteGame() — 적 생성 중단, 완료 로그
패배	GameMgr — curHP 가 0 이하	GameOver() — EnemyMgr.StopGame() 호출로 생성 중단

패배 조건이 성립하지 않는 문제

· 현재 웨이브 설정(3 + 3 + 2 = 적 8마리)과 적 1마리당 HP 감소 10으로는, 모든 적을 놓쳐도 HP 가 20 남는다. 패배가 구조적으로 불가능하다.

· 8절의 밸런싱 검토에서 수치 조정안을 제시한다. 적 수를 최소 25마리로 늘리거나 HP 감소량을 올려야 패배선이 의미를 가진다.

3. 레벨과 맵

3-1. 타일맵 세 장의 역할

맵은 Grid 하나에 타일맵 세 장을 겹쳐 만듭니다. 세 장은 그려지는 그림이 다를 뿐 아니라 게임 규칙 자체가 다릅니다.



그림 3-1 타일맵 세 장의 역할과 Navigation Area 배정

3-2. 맵 설계도

아래는 실제로 타일을 칠할 셀 좌표입니다. 돌길은 직사각형 일곱 개를 Box Fill 로 이어 붙여 만듭니다. 교재는 "그림처럼 그리세요"라고만 하지만, 좌표를 정해 두면 StartPoint 와 EndPoint 가 길 위에 정확히 놓입니다.

맵 설계도 — 셀 좌표로 보는 그림 14-18

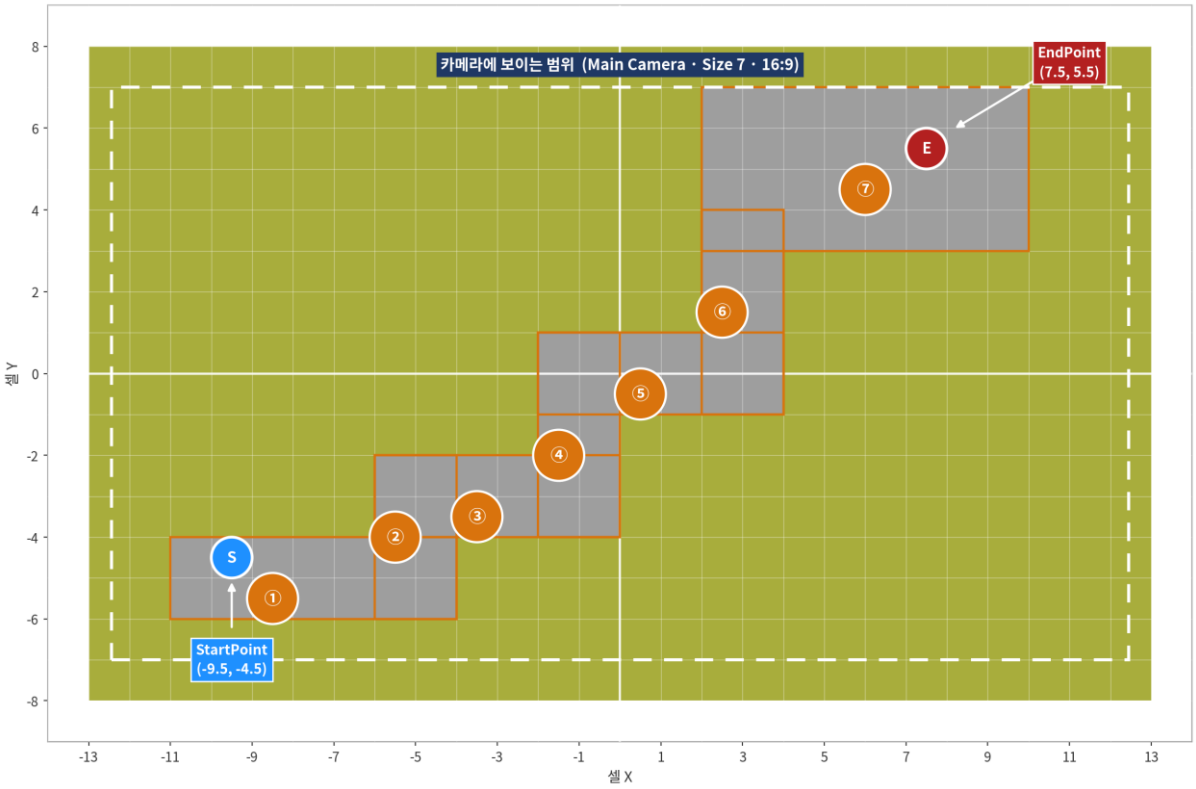


그림 3-2 맵 설계도 — 셀 좌표 기준

순서	셀 범위 (x1, y1) ~ (x2, y2)	설명
배경	(−13, −8) ~ (11, 6)	BgTilemap 에 잔디를 Box Fill. 25 × 15 칸
길 1	(−11, −6) ~ (−5, −5)	시작 구간 — 가로
길 2	(−6, −6) ~ (−5, −3)	첫 번째 꺾임 — 세로
길 3	(−6, −4) ~ (−1, −3)	가로
길 4	(−2, −4) ~ (−1, 0)	세로
길 5	(−2, −1) ~ (3, 0)	가로
길 6	(2, −1) ~ (3, 3)	세로
길 7	(2, 3) ~ (9, 6)	마지막 구간 — 목표 지점

길을 그릴 때의 규칙

· 길 폭은 최소 2칸이다. 1칸이면 Agent Radius 0.2 를 빼고 남는 폭이 너무 좁아 NavMesh 가 중간에 끊긴다.

· 꺾이는 지점에서는 두 직사각형이 반드시 겹치게 그린다. 모서리만 맞닿으면 경로가 이어지지 않는다.

· 돌길을 그릴 자리의 잔디는 지운다. 같은 칸에 잔디(Not Walkable)와 돌길(Walkable)이 겹치면 Not Walkable 이 이겨 길이 통째로 막힌다.

· Building 타일은 길 바로 옆에 붙여 칠한다. 사거리 3 밖에 세운 타워는 한 발도 못 쏜다.

3-3. 좌표계와 단위

항목	값	의미
Pixel Per Unit	32	타일 1칸 = 유니티 1유닛. 모든 거리 수치가 "칸"으로 읽힌다
Grid Cell Size	(1, 1, 0)	2D 타일맵 기본값
Main Camera Size	7	16:9 기준 가로 약 24.9유닛 × 세로 14유닛이 보인다
StartPoint	(−9.5, −4.5, 0)	길 1 위. 적이 생성되는 지점
EndPoint	(7, 5.5, 0)	길 7 위. 적이 도달하면 HP 감소

4. 길 찾기 시스템

유니티의 내비메시는 본래 3D의 바닥면(XZ 평면) 위에 만들어집니다. 2D 게임은 XY 평면을 쓰므로 그대로는 맞지 않습니다. NavMeshPlus 패키지가 이 간극을 메웁니다.

구성요소	하는 일
Navigation Surface	내비메시를 굽는 주체. Bake 버튼이 여기 있다
Navigation CollectSources2d	타일맵·스프라이트에서 모양을 읽어 베이크 재료로 넘긴다. 이것이 없으면 2D 타일맵을 전혀 읽지 못한다
Rotate Surface to XY	CollectSources2d 안의 버튼. 베이크 평면을 −90도 눕혀 XY 평면에 맞춘다. Transform Rotation 이 (−90, 0, 0) 이 된다
Navigation Modifier	타일맵마다 붙여 "이 층은 걸을 수 있다 / 없다"를 표시한다. Override Area 를 켜야 Area 값이 적용된다

4-1. 베이크 순서

이 순서가 아니면 실패합니다. 특히 3번과 6번을 빠뜨리면 베이크해도 파란 영역이 생기지 않습니다.

#	작업	확인할 것
1	git 설치 → Package Manager 로 NavMeshPlus 를 git URL 로 설치 https://github.com/h8man/NavMeshPlus.git	Packages 목록에 보이는가
2	Unity Registry 에서 AI Navigation 설치	같이 설치되어야 Navigation Surface 가 동작한다
3	빈 오브젝트 NavMeshMgr 생성 → Navigation Surface + Navigation CollectSources2d 추가	Position 이 (0, 0, 0) 인가
4	Rotate Surface to XY 클릭	Rotation 이 (−90, 0, 0) 이 되는가
5	타일맵 3장에 Navigation Modifier 설정 Bg: Not Walkable / Obj: Walkable / Building: Not Walkable	Override Area 체크가 켜져 있는가
6	Window > AI > Navigation > Agents 에서 Humanoid 의 Radius 를 0.2 로	기본값 0.5 면 길이 얇은 리본처럼만 깔린다

#	작업	확인할 것
7	씬에 NavMeshAgent 를 가진 오브젝트를 하나 만들어 둔다	Unity 2022.3 이후에서는 이것이 없으면 베이크 결과가 비는 문제가 알려져 있다
8	Clear → Bake	돌길 위에만 파란 영역이 생기는가

교재 순서를 그대로 따르면 막히는 지점

· 교재는 적(NavMeshAgent)을 35쪽에서야 만든다. 그런데 Unity 2022.3 이후에서는 씬에 에이전트가 하나도 없으면 베이크가 비거나 파란 영역이 보이지 않는 문제가 보고되어 있다. 위 표의 7번을 앞으로 당겨야 한다.

· Agent Radius 를 0.2 로 바꾸는 것은 교재 33쪽에 나온다. 32쪽에서 한 번 굽고 33쪽에서 고친 뒤 34쪽에서 다시 굽는 것이 원래 순서인데, 이 설명이 없어 "그림과 다르게 나온다"고 느끼기 쉽다.

· Navigation Surface 의 Advanced 안에 있는 Default Area 가 Walkable 이면, 타일이 없는 빈 여백에도 파란 영역이 생긴다. Not Walkable 로 바꾸면 깨끗해진다.

4-2. 경로가 결정되는 방식

적은 경로를 계산하지 않습니다. NavMeshAgent 에게 목적지 하나만 알려 주면 나머지는 엔진이 합니다.

```
enemyAgent = GetComponent<NavMeshAgent>();

// 2D NavMesh 에서 반드시 필요한 두 줄
enemyAgent.updateRotation = false;   // 에이전트가 진행 방향으로 회전하지 않게
enemyAgent.updateUpAxis   = false;   // Y 축을 위로 삼지 않게 (3D 기본 동작 차단)

enemyAgent.SetDestination(endObject.position);
```

이 두 줄이 없으면 적이 XY 평면 위에서 누워 버리거나 제멋대로 회전합니다. 2D 내비메시를 쓸 때의 필수 설정이며, 교재가 이 부분은 정확히 넣어 두었습니다.

5. 적과 웨이브

5-1. 적의 생애

단계	담당	내용
생성	EnemyMgr.SpawnEnemy()	StartPoint 위치에 NPC 프리팹을 Instantiate. EnemyMove 에 EndPoint 를, Enemy 와 EnemyMove 에 자기 자신을 넘긴다
이동	EnemyMove	NavMeshAgent 가 돌길 위를 따라 EndPoint 로 이동
피격	Enemy.OnDamage()	hpValue 를 깎는다
사망	Enemy → EnemyMgr.DieEnemy()	골드 보상 지급, 리스트에서 제거, 오브젝트 파괴
도달	EnemyMove → EnemyMgr.DieEnemyAtDestination()	GameMgr.damageHP() 호출로 HP 감소, 리스트에서 제거, 오브젝트 파괴

5-2. 도착 판정

목적지에 "도착했다"를 판정하는 조건은 세 가지를 동시에 봅니다.

```
if (!enemyAgent.pathPending &&                      // 경로 계산이 끝났고
    enemyAgent.remainingDistance <= arrivalDistance &&  // 남은 거리가 0.1 이하이며
    (!enemyAgent.hasPath ||                         // 경로가 없거나
     enemyAgent.velocity.sqrMagnitude < sqrVelocityThreshold))  // 거의 멈춰 있다
{
    enemyMgr.DieEnemyAtDestination(gameObject);
    Destroy(gameObject);
}
```

세 조건을 모두 보는 이유는, 경로를 계산하는 동안 remainingDistance 가 잠시 0 으로 보일 수 있기 때문입니다. pathPending 을 함께 확인해야 "출발하자마자 도착 처리"가 일어나지 않습니다.

5-3. 웨이브 데이터 구조

웨이브는 인스펙터에서 편집하는 리스트입니다. 항목 하나가 웨이브 하나에 대응합니다.

```
[System.Serializable]
public class EnemyWaveData
{
    public int   enemyCount;           // 이 웨이브에서 생성할 적의 수
    public float spawnInterval;        // 적 생성 간격(초)
    public float delayBeforeNextWave;  // 다음 웨이브까지의 대기 시간(초)
}
```

웨이브	Enemy Count	Spawn Interval	Delay Before Next Wave
0	3	2 초	1 초
1	3	2 초	2 초
2	2	2 초	3 초

현재 설정값이며, 8절에서 조정안을 제시합니다. 적 8마리로는 패배가 성립하지 않습니다.

5-4. 웨이브 전환 규칙

다음 웨이브로 넘어가는 조건은 세 가지를 모두 만족할 때입니다.

- 이번 웨이브의 적을 전부 생성했다 (isSpawningWave 가 false)
- 살아 있는 적이 하나도 없다 (리스트가 비었다)
- 아직 다음 웨이브를 기다리는 중이 아니다 (중복 예약 방지)

세 조건이 맞으면 delayBeforeNextWave 만큼 쉬었다가 다음 웨이브를 시작합니다. 적이 죽거나 목적지에 도달해 리스트에서 빠질 때마다 이 검사가 돌아갑니다.

6. 타워와 발사체

6-1. 타워 건설

플레이어가 화면을 클릭하면 TowerMgr 이 다음 순서로 처리합니다. 화면 좌표에서 셀 좌표로 두 번 변환하는 것이 핵심입니다.

#	처리	사용하는 것
1	마우스 화면 좌표를 읽는다	Mouse.current.position.ReadValue()
2	화면 좌표 → 월드 좌표	mainCamera.ScreenToWorldPoint()
3	월드 좌표의 z 를 0 으로 맞춘다	2D 이므로. 이걸 빠뜨리면 셀 z 가 어긋나 타일을 못 찾는다
4	월드 좌표 → 셀 좌표	buildTileMap.WorldToCell()
5	이미 타워가 있는 칸인지 확인	Dictionary<Vector3Int, GameObject> 조회
6	그 칸에 Building 타일이 있는지 확인	buildTileMap.GetTile() — null 이면 건설 불가
7	골드가 충분한지 확인	gameMgr.checkGold(towerPrice)
8	셀 좌표 → 월드 좌표, 칸 중앙으로 보정	CellToWorld() + cellSize * 0.5f
9	타워 생성 · 사전에 등록 · 매니저 전달 · 골드 차감	Instantiate → SetEnemeyMgr → consumeGold

여기서 배우는 것

· 화면 좌표 · 월드 좌표 · 셀 좌표는 서로 다른 세계다. 클릭 한 번을 타일 하나로 바꾸려면 두 번의 변환이 필요하다.

· "이미 타워가 있는가"를 타일맵이 아니라 Dictionary 로 기억한다. 타일맵은 타워를 모르기 때문이다. 셀 좌표를 키로 쓰는 것이 자연스러운 선택이다.

· CellToWorld() 는 칸의 모서리를 돌려준다. 칸 중앙에 세우려면 cellSize 의 절반을 더해야 한다.

6-2. 타워의 공격

타워는 FixedUpdate 에서 시간을 누적하다가 공격 간격이 지나면 한 번 쏩니다.

```
curAttackTime += Time.fixedDeltaTime;


if (curAttackTime > waitAttackTime)
{
    findEnemy = enemyMgr.GetClosestEnemy(transform.position, attackRange);
    if (findEnemy != null)
    {
        targetDir = (findEnemy.transform.position - transform.position).normalized;
        // 발사체를 만들고 방향 · 목표 · 매니저를 넘긴다
    }
    curAttackTime = 0.0f;
}
```

적을 찾는 일은 타워가 하지 않고 EnemyMgr 에 맡깁니다. 살아 있는 적의 목록을 EnemyMgr 만 가지고 있기 때문입니다. 타워가 몇 개로 늘어도 탐색 코드는 한 곳에만 있습니다.

```
public GameObject GetClosestEnemy(Vector3 position, float range)
{
    GameObject closest = null;
    float closestSqr = range * range;     // 제곱끼리 비교해 제곱근 연산을 피한다


    foreach (GameObject e in enemyListinCurWave)
    {
        if (e == null) continue;          // 이미 파괴된 적은 건너뛴다
        float sqr = (e.transform.position - position).sqrMagnitude;
        if (sqr <= closestSqr) { closestSqr = sqr; closest = e; }
    }
    return closest;
}
```

6-3. 발사체

항목	내용
이동 방식	목표가 살아 있으면 매 프레임 위치를 다시 읽어 따라간다 (유도). 목표가 죽으면 마지막 위치로 직진하거나 근처의 다른 적으로 갈아탄다
명중 판정	OnTriggerEnter2D — Circle Collider 2D(Is Trigger) + Rigidbody 2D(Gravity Scale 0)
대상 식별	collision.CompareTag("Enemy") — 적 프리팹에 Enemy 태그가 반드시 있어야 한다
소멸 조건	명중 / lifeTime 경과 / 목표가 사라진 자리에 도달
중복 명중 방지	isHit 플래그. Destroy() 는 프레임 끝에 처리되므로 같은 프레임에 두 번 맞을 수 있다

Destroy() 는 즉시 지우지 않는다

· 유니티의 Destroy() 는 호출 즉시 오브젝트를 없애지 않고 그 프레임이 끝날 때 처리한다. 같은 프레임에 발사체 두 발이 맞으면 OnDamage 가 두 번 들어가고, 적이 두 번 죽는 것으로 처리된다.

· isHit 플래그 하나로 막는다. 유니티 입문에서 반드시 짚고 가야 할 개념이다.

7. 경제와 생존

골드와 HP 는 GameMgr 한 곳에 모입니다. EnemyMgr 과 TowerMgr 은 그 값을 읽고 쓰기만 합니다.

자원	증가	감소	한계
골드 (curGold)	적 처치 — addGold(killReward)	타워 건설 — consumeGold(towerPrice)	잔액이 모자라면 건설이 거부된다
HP (curHP)	없음 — 회복 수단이 없다	적이 EndPoint 도달 — damageHP(enemyDamage)	0 이 되면 GameOver()

7-1. 설계상의 비대칭

골드는 늘었다 줄었다 하지만 HP 는 줄기만 합니다. 이 비대칭이 타워 디펜스라는 장르의 긴장을 만듭니다. 플레이어의 실수는 지워지지 않고 끝까지 누적됩니다. 반대로 골드는 회복 가능한 자원이라, 한 번 잘못 세운 타워가 게임을 끝내지는 않습니다.

회복 수단을 추가하는 것은 이 긴장을 푸는 일이므로, 확장 과제에서 다룰 때도 신중히 설계할 필요가 있습니다. 예를 들어 "웨이브를 무실점으로 막으면 HP 1 회복" 같은 조건부 보상이라면 긴장을 유지하면서도 숙련에 대한 보상이 됩니다.

8. 데이터 명세와 밸런싱

8-1. 전체 수치표

스크립트에 들어 있는 모든 조정 가능 수치입니다. 밸런싱은 이 표만 고치면 됩니다.

클래스	필드	현재 값	의미
Enemy	hpValue	100	적의 체력
EnemyMove	arrivalDistance	0.1	목적지 도착으로 판정할 거리
EnemyMove	sqrVelocityThreshold	0.01	멈춤으로 볼 속도의 제곱 (실제 0.1 유닛/초)
EnemyMgr	enemyDamage	10	적 1마리 도달 시 깎이는 HP (제안 수치)
EnemyMgr	killReward	20	적 1마리 처치 시 얻는 골드 (제안 수치)
Tower	attackRange	3.0	사거리 — 3칸
Tower	waitAttackTime	1.0	공격 간격(초)
Projectile	moveSpeed	5.0	발사체 속도 — 초당 5칸
Projectile	attackVal	100.0	발사체 1발의 피해량
Projectile	lifeTime	5.0	발사체 생존 시간(초)
TowerMgr	towerPrice	100.0	타워 1개 건설 비용
GameMgr	curGold	300	시작 골드 — 타워 3개분
GameMgr	curHP	100	시작 체력 — 적 10마리분
NavMeshAgent	Speed	1	적의 이동 속도 — 초당 1칸
Navigation	Agent Radius	0.2	내비메시가 길 가장자리에서 깎아 내는 폭

attackRange 와 waitAttackTime 은 교재 PPT 에서 선언부가 통째로 빠져 있어 값이 명시되지 않았습니다. 위 값은 이 문서가 정한 기준값입니다.

8-2. 밸런싱 검토 — 지금 수치로 계산해 보면

위 수치를 실제로 넣어 계산하면 현재 설정이 게임으로 성립하지 않는다는 것이 드러납니다.

계산	결과	뜻하는 것
경로 전체 길이	약 29칸	적이 Start 에서 End 까지 걷는 데 약 29초

계산	결과	뜻하는 것
타워 1개의 노출 구간	약 6칸 = 6초	사거리 3 의 원을 길이 통과하는 길이
노출 중 발사 횟수	6발	공격 간격 1초
적 1마리를 죽이는 데 필요한 발수	1발	피해 100 vs 체력 100 — 한 방에 죽는다
타워 1개가 막을 수 있는 적	사실상 전부	6발이면 6마리를 연달아 처리한다
전체 적 수	8마리	웨이브 3 + 3 + 2
모두 놓쳤을 때 HP	100 − 80 = 20	패배가 불가능하다

결론 — 세 가지가 동시에 어긋나 있다

· ① 발사체 피해(100)와 적 체력(100)이 같아 모든 적이 한 방에 죽는다. 타워를 여러 개 세울 이유가 없다.

· ② 적이 8마리뿐이라 HP 100 을 깎아 낼 수가 없다. 패배 조건이 작동하지 않는다.

· ③ 처치 보상(20)과 타워 가격(100)의 비율상, 한 판 동안 벌 수 있는 골드가 160 이다. 경제가 돌아가기엔 적 수가 너무 적다.

8-3. 조정안

수업에서 밸런싱을 다룬다면, 아래 수치로 바꾼 뒤 "왜 이렇게 바꿨는가"를 설명하는 순서가 좋습니다. 학생이 직접 조정하게 하는 과제로도 적합합니다.

항목	현재	조정안	근거
Projectile.attackVal	100	25	적 1마리에 4발이 필요해진다. 타워 하나로는 밀려오는 적을 다 막지 못한다
Tower.waitAttackTime	1.0	0.8	발당 피해를 낮춘 만큼 발사 빈도로 보정
웨이브 구성	3 + 3 + 2 = 8마리	8 + 10 + 12 = 30마리	HP 100 ÷ 10 = 10마리 통과가 패배선. 30마리라면 1/3 을 놓치면 진다
EnemyMgr.killReward	20	15	적이 30마리로 늘어 보상 총량이 450 이 된다. 타워 4~5개를 추가로 세울 수 있는 양
Enemy.hpValue	100 고정	웨이브마다 ×1.3	후반 웨이브에 압박을 만든다. 난이도 곡선이 생긴다

조정안대로라면 한 판은 약 3분, 플레이어는 타워를 6~8개까지 세우게 되고, 마지막 웨이브에서 HP 가 실제로 위험해집니다.

9. 씬 구성과 오브젝트 명세

9-1. 하이어라키

```
Main2DDefenseGameScene
├─ Main Camera                Size 7
├─ Global Light 2D
├─ Grid
│   ├─ BgTilemap              Order in Layer 0 · Navigation Modifier → Not Walkable
│   ├─ ObjTilemap             Order in Layer 1 · Navigation Modifier → Walkable
│   └─ Building               Order in Layer 1 · 초록 틴트 · Not Walkable
├─ StartPoint                 (-9.5, -4.5, 0)   빈 오브젝트
├─ EndPoint                   (7, 5.5, 0)       빈 오브젝트
├─ NavMeshMgr                 Navigation Surface + Navigation CollectSources2d
├─ CurEnemyMgr                EnemyMgr.cs
├─ CurTowerMgr                TowerMgr.cs
└─ CurGameMgr                 GameMgr.cs
```

9-2. 프리팹 3 종

프리팹	모양 · 크기	컴포넌트	비고
NPC (적)	원 스프라이트 · Scale 0.5 빨강 · Order in Layer 2	NavMeshAgent (Speed 1) Circle Collider 2D (R 0.5, Is Trigger) Enemy.cs · EnemyMove.cs	Tag 를 반드시 Enemy 로. 발사체가 CompareTag 로 판정한다
TowerObj (타워)	사각 스프라이트 노랑 · Order in Layer 3	Tower.cs	projectTile 속성에 Projectile 프리팹을 할당
Projectile (발사체)	원 스프라이트 · Scale 0.2 파랑 · Order in Layer 3	Circle Collider 2D (R 0.2, Is Trigger) Rigidbody 2D (Gravity Scale 0) Projectile.cs	Rigidbody 가 있어야 Trigger 가 동작한다

9-3. 인스펙터 연결

이 표의 연결이 하나라도 비면 게임이 조용히 멈춥니다. 체크리스트로 쓰기 좋은 부분입니다.

컴포넌트	속성	넣는 것
EnemyMgr (CurEnemyMgr)	Start Point / End Point	StartPoint / EndPoint 오브젝트
EnemyMgr	Enemy Prefab	Prefabs/NPC
EnemyMgr	Waves	[+] 로 웨이브 항목 추가

컴포넌트	속성	넣는 것
EnemyMgr	Game Mgr	CurGameMgr 오브젝트
Tower (프리팹)	Project Tile	Prefabs/Projectile
TowerMgr (CurTowerMgr)	Build Tile Map	Building 타일맵 — BgTilemap 이 아님
TowerMgr	Build Tower	Prefabs/TowerObj
TowerMgr	Enemy Mgr	CurEnemyMgr 오브젝트
TowerMgr	Game Mgr	CurGameMgr 오브젝트

10. 스크립트 아키텍처

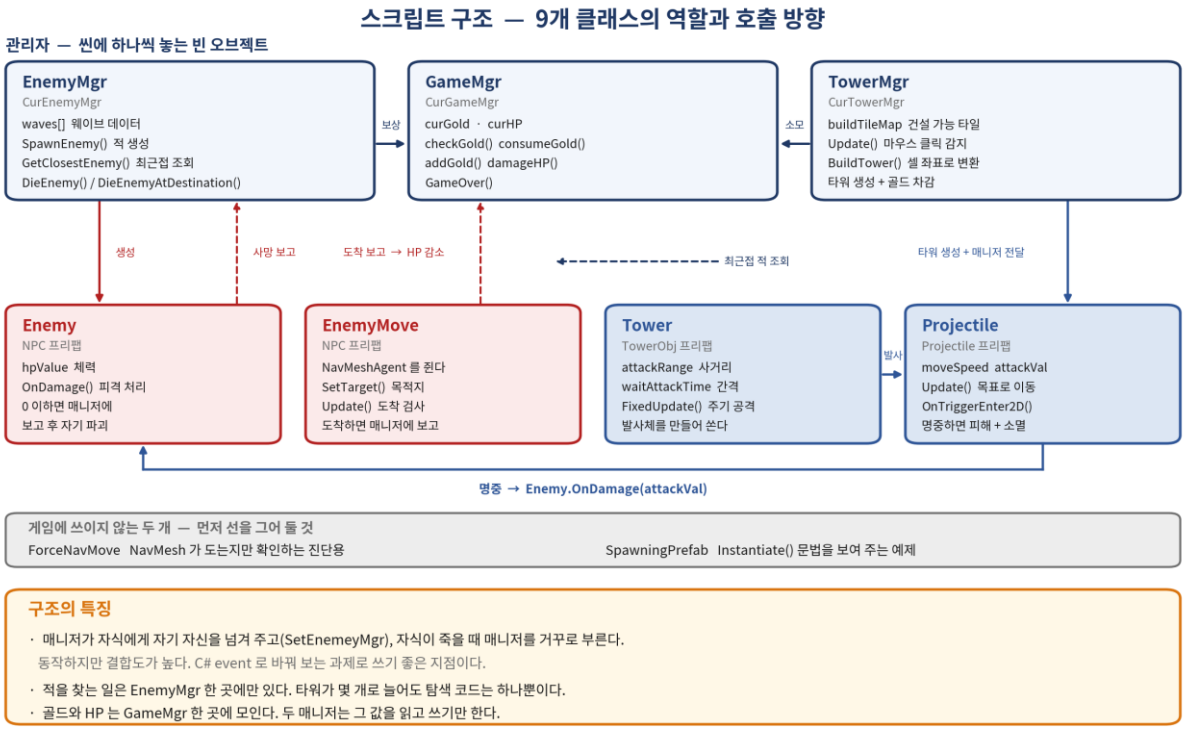


그림 10-1 9개 클래스의 역할과 호출 방향

10-1. 파일 목록

파일	붙는 곳	역할	교재 코드
EnemyMgr.cs	CurEnemyMgr	웨이브 관리 · 적 생성 · 최근접 적 조회 · 적의 퇴장 처리	14-4
Enemy.cs	NPC 프리팹	체력과 피격, 사망 보고	14-2
EnemyMove.cs	NPC 프리팹	NavMeshAgent 이동과 도착 판정	14-3
TowerMgr.cs	CurTowerMgr	마우스 클릭 → 셀 좌표 → 타워 건설	14-7
Tower.cs	TowerObj 프리팹	사거리 안 최근접 적을 주기마다 공격	14-5
Projectile.cs	Projectile 프리팹	발사체 이동과 명중 처리	14-6
GameMgr.cs	CurGameMgr	골드와 HP 관리, 승패 판정	14-8
ForceNavMove.cs	테스트용	NavMesh 동작 확인 전용 — 게임에 쓰이지 않음	14-1
SpawningPrefab.cs	예제용	Instantiate 문법 예제 — 게임에 쓰이지 않음	—

10-2. 실행 순서

한 판이 시작될 때 일어나는 일의 순서입니다. 유니티의 생애주기 함수가 어떤 순서로 불리는지 이해하는 데 좋은 예가 됩니다.

순서	일어나는 일
1	EnemyMgr.Start() — 인스펙터 연결을 검사하고 StartGame() 호출
2	TowerMgr.Start() — mainCamera 를 Camera.main 으로 채운다
3	EnemyMgr.SpawnEnemyRoutine() — 0번 웨이브 시작, 첫 적을 즉시 생성
4	InvokeRepeating 으로 spawnInterval 마다 SpawnEnemy() 반복
5	생성된 적의 EnemyMove.Start() — 다음 프레임에 실행되어 SetDestination 호출
6	매 프레임 TowerMgr.Update() 가 클릭을 감시, Tower.FixedUpdate() 가 공격 타이밍을 센다

Awake 와 Start 의 차이가 여기서 중요해진다

· EnemyMgr 은 Instantiate 직후에 enemyMove.SetTarget(endPoint) 을 호출한다. 이것이 통하는 이유는 생성된 오브젝트의 Start() 가 다음 프레임에 돌기 때문이다.

· 만약 EnemyMove 가 Start() 가 아니라 Awake() 에서 SetDestination 을 했다면, Awake() 는 Instantiate() 안에서 즉시 실행되므로 SetTarget 보다 먼저 돌아 목적지가 null 인 채로 끝난다.

· 생애주기 함수를 고르는 일이 왜 설계 결정인지 보여 주는 지점이다.

10-3. 구조의 특징과 한계

매니저가 자식에게 자기 자신을 넘겨 주고(SetEnemeyMgr), 자식이 죽을 때 매니저를 거꾸로 호출하는 구조입니다. 동작하지만 결합도가 높습니다. Enemy 는 EnemyMgr 이라는 구체적인 클래스를 알아야만 하고, EnemyMgr 없이는 테스트할 수 없습니다.

C# 의 event 로 바꾸면 Enemy 는 "나 죽었다"고 알리기만 하고 누가 듣는지는 모르게 됩니다. 13절의 확장 과제에 넣어 두었습니다.

11. 개발 단계와 체크포인트

각 단계의 끝에 "화면이 이렇게 보여야 한다"를 두었습니다. 여기서 막히면 다음으로 넘어가지 않는 편이 결국 빠릅니다.

단계	작업	체크포인트
0. 준비	Unity 6.3 · 패키지 3종(NavMeshPlus · AI Navigation · Input System) · 에셋 임포트 · 폴더 구조	Package Manager 에 세 개가 모두 보이는가
1. 맵	스프라이트 임포트(PPU 32, Point, Multiple) → 32×32 슬라이스 → 타일 팔레트 → 타일맵 3장 → 길 그리기	돌길이 시작점에서 목표점까지 끊김 없이 이어졌는가
2. 길 찾기	Navigation Modifier 3개 → NavMeshMgr → Rotate Surface to XY → Agent Radius 0.2 → Bake	파란 영역이 돌길 위에만, 끊김 없이 깔렸는가
3. 적	NPC 프리팹(Tag Enemy 포함) → Enemy.cs · EnemyMove.cs → StartPoint · EndPoint	적 하나가 길을 따라 끝까지 걸어가는가
4. 웨이브	EnemyMgr.cs → waves 설정 → 생성 간격과 웨이브 전환	적이 줄지어 나오고, 다 죽으면 다음 웨이브가 오는가
5. 타워	Building 타일맵 → TowerObj 프리팹 → Tower.cs → Projectile 프리팹 → TowerMgr.cs	초록 타일을 클릭하면 타워가 서고, 발사체가 적을 맞히는가
6. 경제	GameMgr.cs → 골드 차감 · 처치 보상 · HP 감소 · 승패 판정	골드가 줄고 늘며, HP 가 0 이 되면 게임이 끝나는가
7. 밸런싱	8절의 조정안 적용 → 플레이테스트	한 판이 2~3분이고, 지는 경우가 실제로 생기는가

12. 알려진 제약과 미구현

12-1. 미구현 기능

기능	상태	비고
UI	전혀 없음	골드 · HP · 웨이브 번호가 화면에 표시되지 않는다. Console 로만 확인 가능
사운드	없음	발사 · 명중 · 건설 효과음 없음
타워 업그레이드	없음	교재 01절이 구성요소로 들었으나 구현 없음
타워 종류	1종	사거리 · 공격 속도 · 피해량이 다른 변형 없음
적 종류	1종	체력 · 속도가 다른 변형 없음
타워 판매 · 이동	없음	한 번 세우면 되돌릴 수 없다
일시정지 · 재시작	없음	씬을 다시 불러오는 방법뿐
저장	없음	최고 기록 등이 남지 않는다

12-2. 교재 원본 코드의 결함

출판사 제공 PPT 의 코드를 그대로 입력하면 프로젝트가 컴파일되지 않습니다. 교재를 그대로 쓸 때 반드시 보완해야 하는 부분입니다.

파일	결함	영향
EnemyMgr.cs	클래스를 닫는 중괄호 누락 · Start() 없음 · CompleteGame / DieEnemy / DieEnemyAtDestination / GetClosestEnemy 정의 없음 · 범위 검사 순서 역전 · 반복 조건 off-by-one	컴파일 불가. 고쳐도 웨이브가 시작되지 않는다
EnemyMove.cs	SetTarget() 이 받은 값을 저장하지 않는다	적이 생성되어도 목적지를 모른 채 제자리에 선다
Tower.cs	필드 선언 05~13행이 통째로 빠짐 · targetDir 을 계산하는 코드 없음	컴파일 불가. 고쳐도 발사체가 움직이지 않는다
Projectile.cs	Update · SetDirection · SetTarget · SetEnemeyMgr 전부 생략	Tower 쪽에서 컴파일 오류 3건

파일	결함	영향
TowerMgr.cs	Start / Update 생략 · checkGold 호출에 gameMgr. 누락 · consumeGold 미호출 · towerCounter 미증가	컴파일 불가. 고쳐도 클릭이 아무 일도 하지 않는다
GameMgr.cs	consumeGold 잔액 검사 없음 · damageHP 중복 GameOver · 호출하는 쪽이 없음	골드가 음수로 내려가고, HP 가 줄지 않는다
공통	Input System 설치 단계 없음 · 적 프리팹의 Enemy 태그 설정 단계 없음	TowerMgr 컴파일 오류 / 발사체가 적을 통과한다

12-3. 기술적 제약

- **WebGL 빌드에서 NavMesh:** 베이크된 내비메시 데이터는 빌드에 포함되므로 WebGL 에서도 동작한다. 다만 빌드 용량이 커지고 첫 로딩이 길어진다.
- **모바일:** 마우스 클릭(Mouse.current)을 전제로 하므로 터치 입력이 동작하지 않는다. 모바일 지원은 Touchscreen 처리를 따로 넣어야 한다.
- **성능:** 적과 발사체를 Instantiate / Destroy 로 매번 만들고 지운다. 적이 수십 마리로 늘어나면 오브젝트 풀링이 필요해진다.
- **GetClosestEnemy 의 비용:** 타워마다 매 공격 주기에 전체 적 목록을 훑는다. 타워 N 개 × 적 M 마리의 비교가 일어난다. 소규모에서는 문제없지만 구조적으로는 공간 분할이 필요한 지점이다.

13. 확장 과제

기본 구현을 마친 뒤 학생이 직접 붙여 볼 과제입니다. 난이도 순으로 정렬했고, 각 항목에 "어느 파일을 건드리는가"를 함께 적었습니다.

난이도	과제	건드리는 곳	배우는 것
하	골드 · HP · 웨이브 번호를 화면에 표시	Canvas + UI Text, GameMgr	UI 와 로직의 분리
하	발사 · 명중 · 건설 효과음	AudioSource, 각 스크립트	AudioSource.PlayOneShot
하	타워 사거리를 마우스 올렸을 때 원으로 표시	Tower + LineRenderer	시각적 피드백의 가치
중	타워 종류 2~3개 (사거리형 · 속사형)	Tower 를 상속하거나 ScriptableObject 로 수치 분리	데이터와 로직의 분리
중	적 종류 2~3개 (빠른 적 · 단단한 적)	NPC 프리팹 변형, EnemyWaveData 에 종류 추가	프리팹 변형과 데이터 설계
중	타워 업그레이드 (골드를 더 내고 강화)	Tower 에 레벨, TowerMgr 에 클릭 분기	같은 입력의 다중 의미
중	웨이브마다 적 체력 증가	EnemyMgr 에서 생성 직후 hpValue 조정	난이도 곡선 설계
상	타워 판매 · 이동	TowerMgr 의 Dictionary 관리	자료구조의 일관성 유지
상	적 사망 · 도달을 C# event 로 바꾸기	Enemy · EnemyMove · EnemyMgr	결합도 낮추기
상	오브젝트 풀링 적용	적과 발사체 생성 전반	메모리와 GC, 재사용 패턴

13-1. 추천 — 먼저 할 것 세 개

시간이 한정되어 있다면 다음 셋을 권합니다. 적은 노력으로 게임의 체감이 가장 크게 달라지는 항목들입니다.

- **UI 표시:** 골드와 HP 가 보이지 않으면 플레이어는 자기가 무엇을 하고 있는지 알 수 없다. 지금 이 게임이 "게임처럼 느껴지지 않는" 가장 큰 이유다.
- **타워 사거리 표시:** 어디에 세울지 고르는 것이 이 게임의 핵심 결정인데, 사거리가 보이지 않으면 그 결정을 눈으로 할 수 없다.

- **밸런싱 조정:** 8 절의 조정안을 적용하는 것만으로 "질 수 있는 게임"이 된다. 질 수 없는 게임은 이길 때도 기쁘지 않다.

14. 참고 자료

자료	출처
한빛아카데미『유니티 입문』14장「실전 2D 게임 프로젝트」	교재 본문 및 출판사 제공 PPT 65쪽
NavMeshPlus (h8man) — 2D NavMesh 패키지	https://github.com/h8man/NavMeshPlus
NavMeshPlus 사용법	https://github.com/h8man/NavMeshPlus/wiki/HOW-TO
Unity 2022.3 이후 베이크 실패 사례	https://github.com/h8man/NavMeshPlus/issues/245
Pixel Art Top Down – Basic (TX Tileset Grass)	Unity Asset Store

이 문서는 원광대학교 게임콘텐츠학과 교육 목적으로 작성되었습니다. 교재 내용의 저작권은 한빛아카데미에 있으며, 본 문서는 수업에서 사용하기 위한 분석·재구성 자료입니다.